ISB30503

DATA STRUCTURE & ALGORITHM

TEAM**MEMBERS**

KHAIRUL NAQIB BIN KUSHAINI
52213123278

AKMAL HAKIMI BIN ABD RASHID
52213123466

SYED MUHAMMAD AKIL BIN SYED ALI
52213123249

BATRISHA ELYANI FAKHIRA BINTI ROSLEE
52213124203

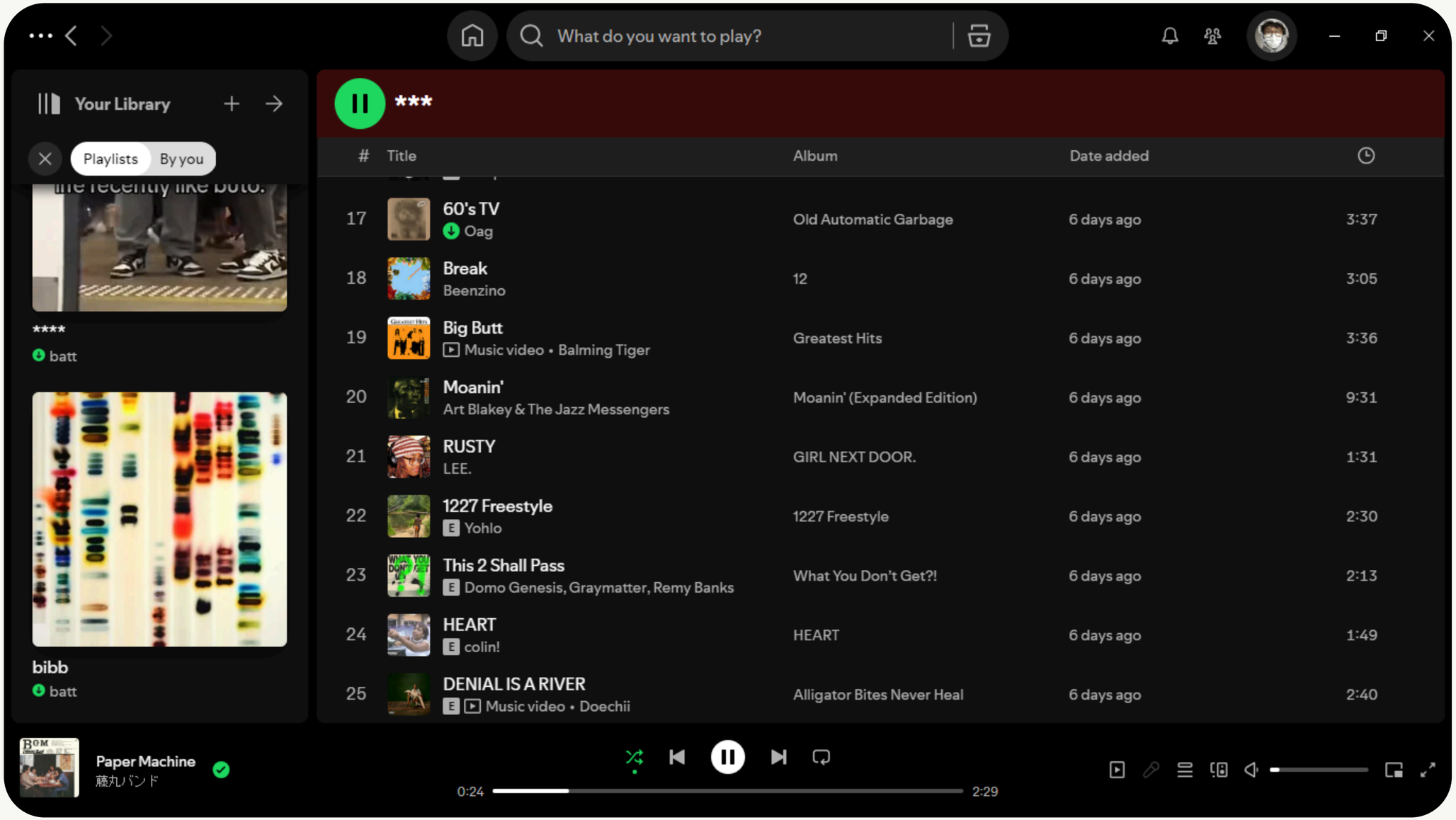
APPLICATION **TITLE**

IMPLEMENTATION OF LINKEDLIST IN SPOTIFY

SPOTIFY IS A DIGITAL MUSIC, PODCAST, AND VIDEO STREAMING SERVICE THAT ALLOWS USERS TO ACCESS MILLIONS OF SONGS AND OTHER CONTENT. IT'S AVAILABLE ON MANY DEVICES, INCLUDING PHONES, COMPUTERS, TABLETS, AND SMART HOME DEVICES.

IN SPOTIFY, LINKED LISTS ARE A POTENTIAL DATA STRUCTURE THAT COULD BE UTILIZED FOR EFFICIENT MANAGEMENT OF PLAYLISTS, QUEUES, OR MUSIC TRACKS.

SPOTIFY PLAYLIST **INTERFACE**



DATA STRUCTURE & ALGORITHM

HOW SPOTIFY WORK
SPOTIFY OPERATIONS

ADD A SONG

Adds a new song to the playlist at the end or a specific position.

REMOVE SONG

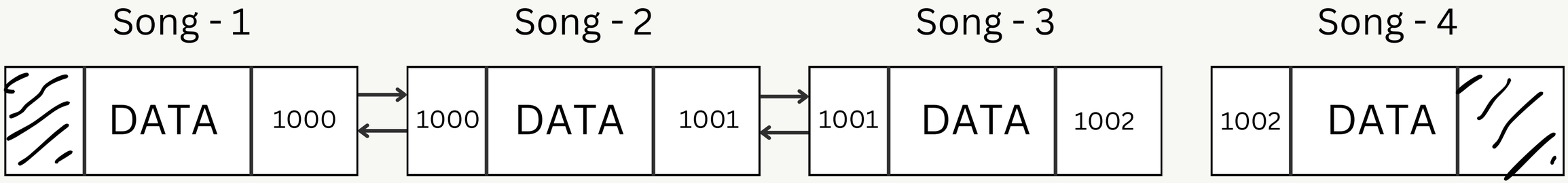
Deletes a song by its position or name.

FORWARD SONG

Moves to the next song in the playlist.

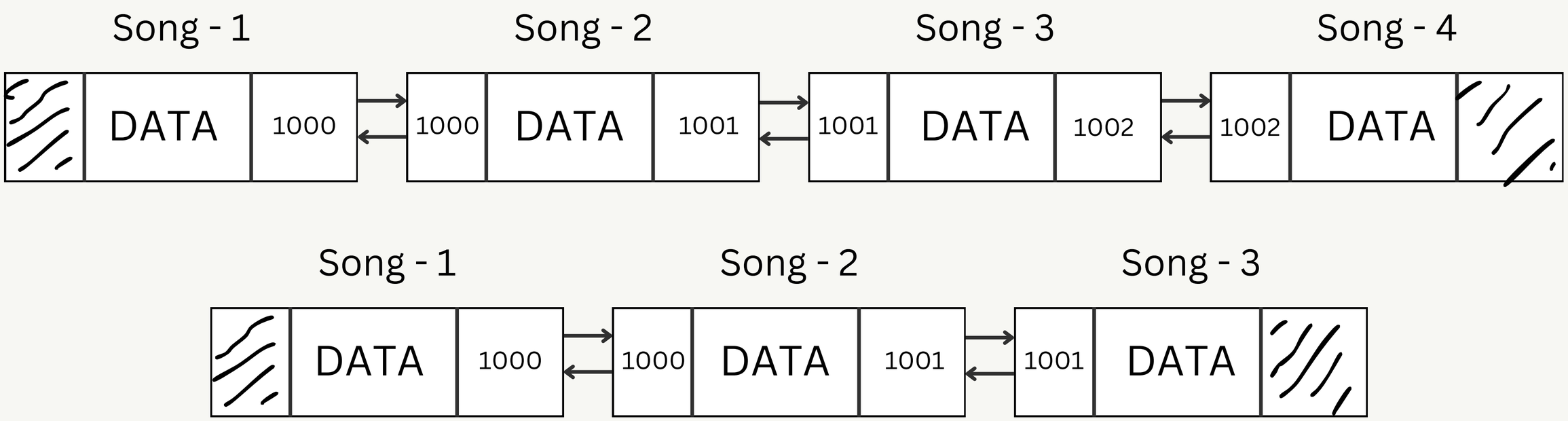
STORAGE PATTERN

1. ADD SONG



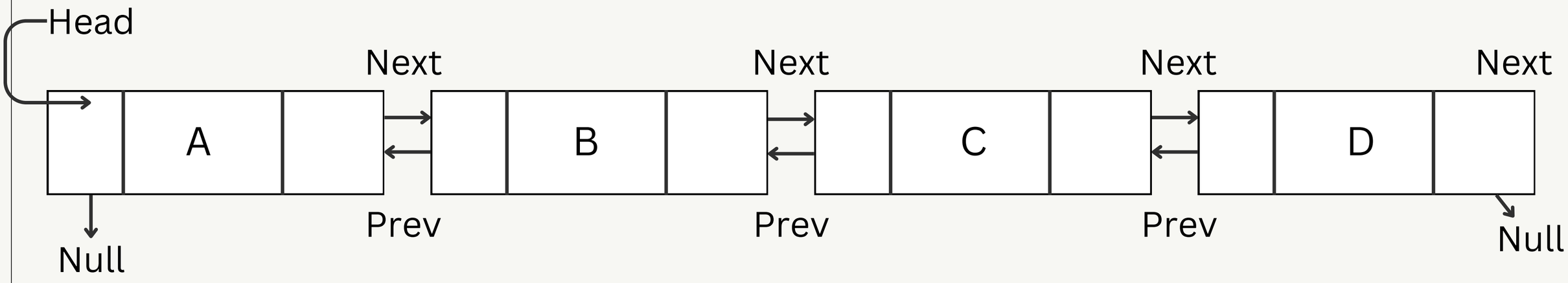
STORAGE PATTERN

1. REMOVE SONG



STORAGE PATTERN

1. FORWARD SONG



RUNNING TIME

EFFICIENCY OF THE ALGORITHM

ADD SONG

ALGORITHM PSEUDOCODE

```
ALGORITHM AddSongToPlaylist(playlist, song, position=None)
    IF position IS NULL:
        Append song to playlist
    ELSE:
        IF position < 0 OR position > len(playlist):
            THROW "Invalid position"
        INSERT song INTO playlist AT position
    RETURN playlist
```

RUNNING TIME

EFFICIENCY OF THE ALGORITHM

ADD SONG

RUNNING TIME ANALYSIS

1. Analyze Key Operations:

- Case 1: Adding to the End (Position is Null)
- The append operation in an ArrayList takes O(1) time as the element is added to the end of the array.
- Best Case Time Complexity: O(1).
- Case 2: Adding at a Specific Position (Position is Given):
- Validating the position takes O(1).
- Insertion:
- Shifting elements from the insertion point to the end of the array takes O(n), where n is the size of the playlist.
- Worst Case Time Complexity: O(n) (when inserting at the beginning or in a large playlist).

2. Space Complexity:

- Since the ArrayList is dynamic, no extra memory is allocated apart from the existing array, making the space complexity O(1).

Steps for Running Time Estimation:

1. Assume a playlist with n elements.
2. For appending, 1 operation is performed (O(1)).
3. For insertion:

- 1 validation operation: O(1).
- Shifting elements: O(n).

RUNNING TIME

ADD SONG

EFFICIENCY OF THE ALGORITHM

EFFICIENCY OF THE ALGORITHM

- The efficiency depends on the operation type:
- Appending: Efficient with constant time (O(1)).
- Inserting: Less efficient with linear time (O(n)), particularly for large playlists.

RUNNING TIME

EFFICIENCY OF THE ALGORITHM

ADD SONG

CONCLUSION

Findings:

- The addSongToPlaylist method is efficient for appending songs but less efficient for inserting songs at specific positions due to the need for shifting elements in the ArrayList.
- • For large playlists, insertion at the beginning or middle can result in significant overhead.
- • Recommendation:
- • Use LinkedList instead of ArrayList if frequent insertions at arbitrary positions are required, as it offers O(1) insertion at any position.

WHY LINKEDLIST

PURPOSE

- Doubly Linked List: Enables bidirectional navigation for seamless movement between songs.
- Efficient Insertions and Deletions: No need to shift elements like in arrays.
- Dynamic Resizing: Memory is allocated only when a new song is added.

LINKEDLIST

DOUBLY LINKED LIST STRUCTURE

next: Points to the next node.
prev: Points to the previous node.

EXISTING APPLICATION'S CODE

MAIN

```java
public class Main {
    public static void main(String[] args) {
        Playlist playlist = new Playlist();

        System.out.println("Adding songs to the playlist...");
        playlist.addSong(new Song("Imagine", "John Lennon"));
        playlist.addSong(new Song("Bohemian Rhapsody", "Queen"));
        playlist.addSong(new Song("Highway to Hell", "AC/DC"));
        playlist.addSong(new Song("Billie Jean", "Michael Jackson"));

        System.out.println("\nCurrent Playlist:");
        playlist.displayPlaylist();
    }
}
```

EXISTING APPLICATION'S CODE

SONG

```java
public class Song {
    private String title;
    private String artist;

    public Song(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }

    public String getTitle() {
        return title;
    }

    public String getArtist() {
        return artist;
    }

    @Override
    public String toString() {
        return "\"" + title + "\" by " + artist;
    }
}
```

EXISTING APPLICATION'S CODE

PLAYLIST

```java
public class Playlist {
    private class Node {
        Song song;
        Node next;
        Node prev;

        Node(Song song) {
            this.song = song;
        }
    }

    private Node head;
    private Node tail;

    // Add a song to the end of the playlist
    public void addSong(Song song) {
        Node newNode = new Node(song);
        if (tail == null) { // Empty list
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    // Display all songs in the playlist
    public void displayPlaylist() {
        if (head == null) {
            System.out.println("Playlist is empty.");
            return;
        }
        Node current = head;
        int index = 1;
        while (current != null) {
            System.out.println(index + ". " + current.song);
            current = current.next;
            index++;
        }
    }
}
```

COLLECTION CLASS

Class Name	Purpose	Attributes
Playlist	Implements the doubly linked list for managing a Spotify playlist.	- Node head: Points to the start of the list. - Node tail: Points to the end of the list.
Node	Represents a node in the doubly linked list containing a song and pointers to adjacent nodes.	- Song song: The song data in the node. - Node next: Points to the next node. - Node prev: Points to the previous node.
Song	Represents a single song in the playlist.	- String title: The title of the song. - String artist: The artist of the song.

INTERFACES USED

Interface Name	Purpose	Operations Provided
Java Object	Implicit superclass for all Java classes, providing default functionality like toString().	- toString(): Converts objects into a string format. - Other implicit methods (e.g., equals, hashCode).

LIST OF OPERATIONS

PLAYLIST CLASS

Operation Name	Description
addSong(Song song)	Adds a new song to the end of the playlist. Adjusts the next and prev pointers of adjacent nodes.
displayPlaylist()	Iterates through the linked list and prints the details of each song.

LIST OF OPERATIONS

NODE CLASS

Operation Name	Description
Node(Song song)	Constructor to create a node containing a song.
next	Pointer to the next node in the linked list.
prev	Pointer to the previous node in the linked list.

LIST OF OPERATIONS

SONG CLASS

Operation Name	Description
Song(String title, String artist)	Constructor for initializing the song object with title and artist.
getTitle()	Returns the title of the song.
getArtist()	Returns the artist of the song.
toString()	Overrides the toString() method to return a formatted string representation of the song.

EXAMPLE OUTPUT

OUTPUT

```
Adding songs to the playlist...

Current Playlist:
1. "Imagine" by John Lennon
2. "Bohemian Rhapsody" by Queen
3. "Highway to Hell" by AC/DC
4. "Billie Jean" by Michael Jackson
```

CONCLUSION

- This project highlights an efficient implementation of linked lists to manage Spotify playlists, ensuring dynamic and seamless song navigation.
- Linked lists, particularly the doubly linked list structure, enable efficient insertions and deletions with O(1) complexity, eliminating the need for shifting elements as seen in ArrayLists.
- The implementation leverages classes like Playlist, Node, and Song to organize and manage data systematically, supporting real-world playlist operations.
- ArrayLists are effective for appending songs with O(1) complexity but become less efficient with O(n) complexity for insertions at arbitrary positions, especially in large playlists.
- Linked lists are recommended for dynamic applications like Spotify playlists, where frequent updates are necessary, providing a practical and scalable solution.